

Aula 9

Virando o cartão com o clique

► **Unidade**

**Página web: desenvolvendo uma
ferramenta interativa de estudo**

O que vamos aprender?

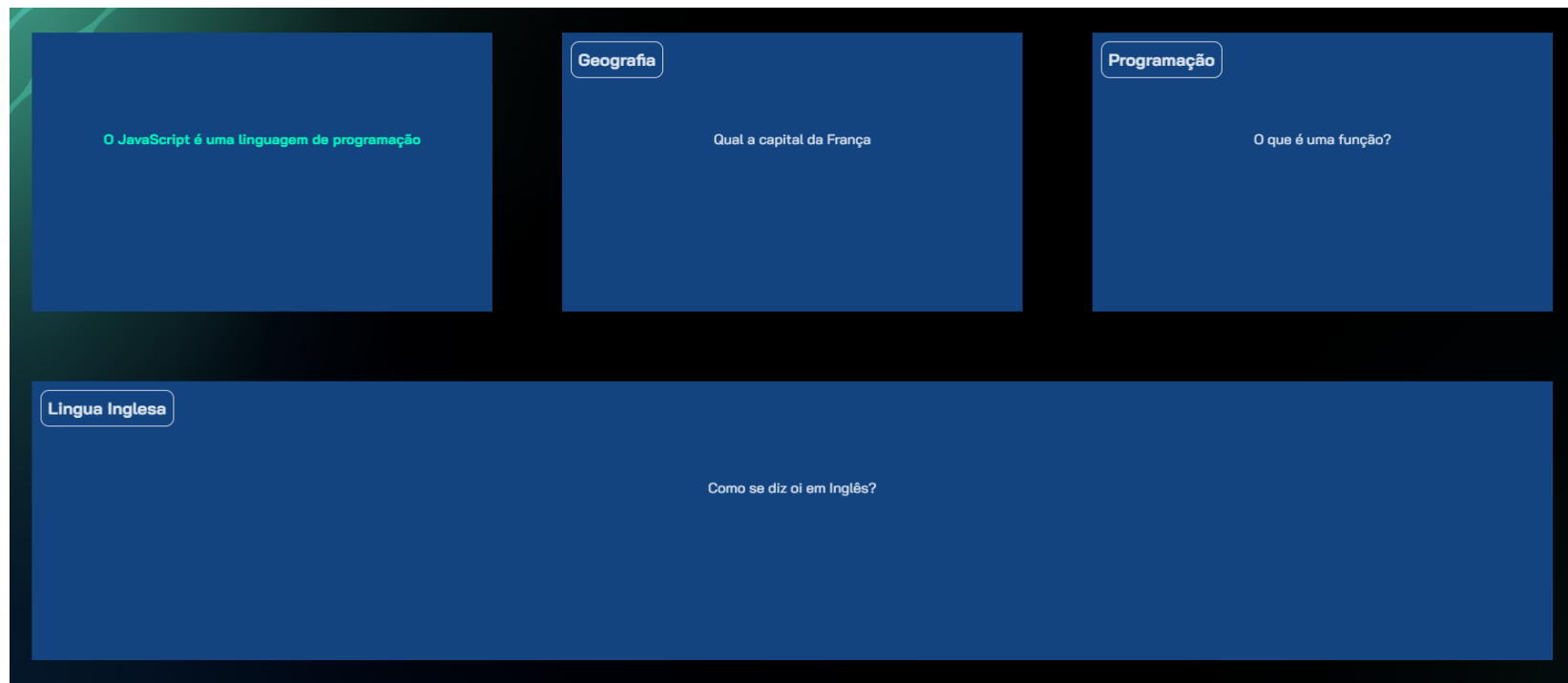
-  Modificar o giro dos cartões.
-  Compreender o uso de um operador lógico.
-  Utilizar a função **`addEventListener`**.



 CLIQUE E ACOMPANHE A AULA NA ALURA

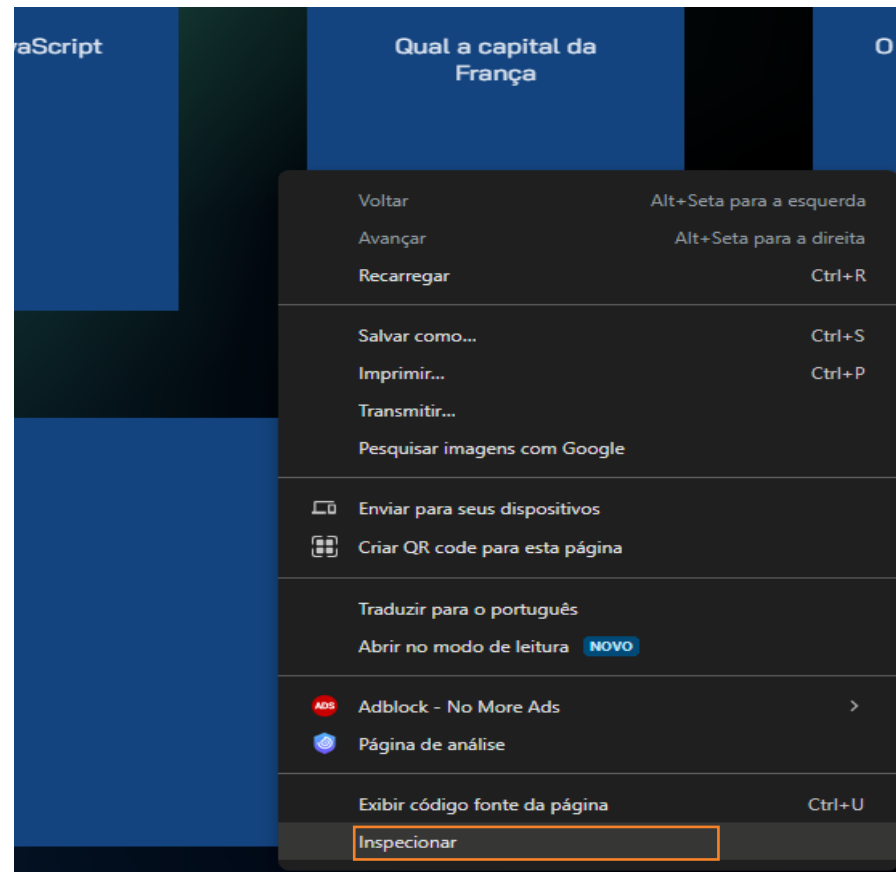
O método `addEventListener()`

Na aula anterior, utilizamos o conceito de função, em JavaScript, para exibir diferentes perguntas na nossa página sem a necessidade de adicioná-las manualmente no HTML. Agora, faremos uma nova modificação para que o verso do cartão, que contém a resposta, seja exibido apenas quando clicarmos sobre o card.

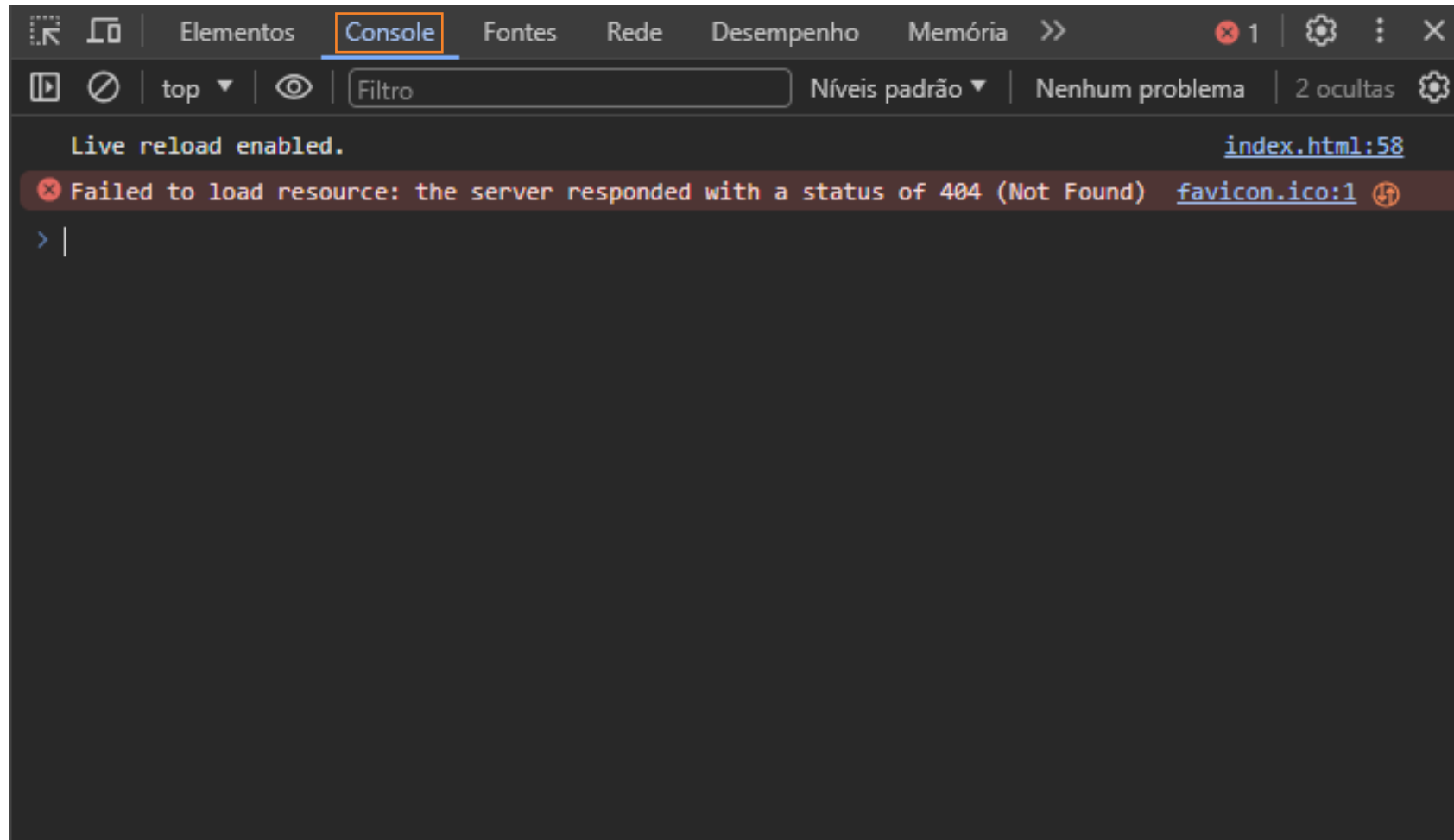


Queremos que a rotação dos cartões passe a ser acionada com o clique do mouse, em vez de apenas quando passamos o mouse sobre eles. Para isso, começaremos analisando nossa página por meio do recurso de *Inspecionar*, que já conhecemos.

Na página do seu projeto, clique com o botão direito do mouse e selecione a opção *Inspecionar*, conforme imagem a seguir:



Uma seção na lateral se abrirá na página. Nela, precisaremos acessar o console, no qual poderemos realizar alguns testes. Veja na imagem abaixo onde você fará isso:



Vamos começar nossos testes digitando comandos nesse espaço.

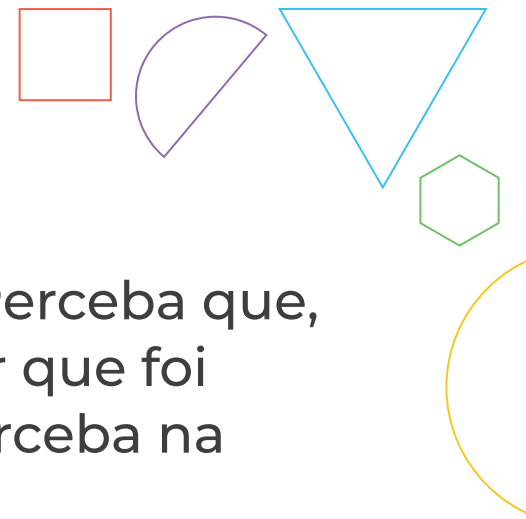


A primeira coisa que precisamos perceber é que, quando acessamos a página, a resposta não fica visível. Isso acontece apenas quando passamos o mouse sobre algum cartão. Para começarmos a perceber isso, criaremos uma variável chamada **respostaEstaVisivel** e diremos que essa variável é igual a **false**, pois a resposta não está visível. O código, no console, ficará da seguinte forma:

```
let respostaEstaVisivel = false
```

Para dizer que a resposta está visível, faremos o oposto disso, atribuindo o valor **true**. Observe:

```
let respostaEstaVisivel = true
```



Você pode escrever uma linha abaixo da outra no console. Perceba que, ao digitarmos o nome da variável novamente, o último valor que foi guardado nela é apresentado, que, nesse caso, é o **true**. Perceba na imagem como isso aparece no console:

```
> let respostaEstaVisivel = false
< undefined
> let respostaEstaVisivel = true
< undefined
> respostaEstaVisivel
< true
```



Agora, precisamos compreender como controlar essa alteração entre **true** e **false** para fazer os cartões exibirem a resposta no momento correto. Para isso, utilizaremos uma estrutura chamada de operador lógico ou negação. Com esse operador, conseguimos inverter o valor atribuído à variável, de modo que ele mude de **true** para **false** e vice-versa, de acordo com o nosso comando. No código, ele é escrito da seguinte forma, utilizando o sinal de exclamação ! :

```
respostaEstaVisivel = !respostaEstaVisivel
```

Se digitamos esse operador no console e apertamos a tecla *Enter*, veremos que o valor da variável é invertido. Veja como isso é exibido no console:

```
> let respostaEstaVisivel = true
< undefined
> respostaEstaVisivel
< true
> respostaEstaVisivel = !respostaEstaVisivel
< false
> respostaEstaVisivel = !respostaEstaVisivel
< true
> |
```

Assim, cada vez que utilizarmos o operador lógico, o valor da variável será trocado. A partir disso, conseguiremos manipular a forma como exibimos a nossa resposta.



Agora, vamos ao VS Code. Iniciaremos criando a variável em nosso arquivo *app.js*. Lembre-se de que ela começa com o valor **false**, pois, no início da aplicação, as respostas nunca são exibidas. O código ficará assim:

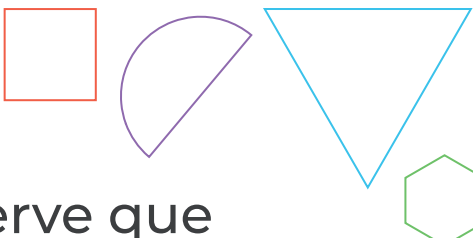
```
let respostaEstaVisivel = false
container.appendChild(cartao)
```

Perceba que criamos a variável no final do arquivo *app.js*. Agora, precisamos programar a ação de virar o cartão. Quando precisamos criar um conjunto de ações para realizar uma tarefa, utilizamos a estrutura que aprendemos na aula anterior: uma função. Dessa forma, criaremos a função de virar o cartão logo abaixo da declaração da variável:

```
let respostaEstaVisivel = false

function viraCartao () {
  respostaEstaVisivel = !respostaEstaVisivel
}
```

Dentro da função, utilizaremos o operador lógico para inverter o valor da variável. Nesse caso, se ele for verdadeiro, ela se tornará falsa, e vice-versa.



Agora, precisamos fazer uma alteração no arquivo CSS. Observe que utilizamos a propriedade **hover** para indicar que a ação de virar o cartão será acionada ao passar o mouse sobre ele. Portanto, mudaremos de **hover** para **active**.

O código está assim:

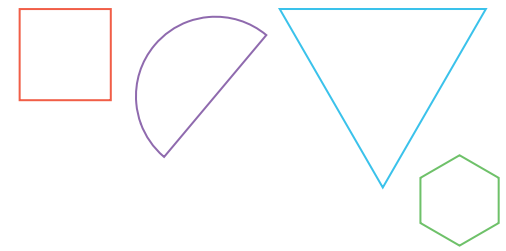
```
.cartao: hover .cartao__conteudo {  
  transform: rotateY(180deg);  
}
```

E deverá ficar assim:

```
.cartao.active .cartao__conteudo {  
  transform: rotateY(180deg);  
}
```

Perceba que, para utilizar a função **active**, utilizamos o sinal de ponto-final antes dela.

Ao salvar o projeto e testá-lo, veremos que os cartões não giram mais.



Para corrigir isso, precisamos ativar a função responsável por girar o cartão, que é a função **active**. Observe:

```
function viraCartao () {  
  respostaEstaVisivel = !respostaEstaVisivel  
  cartao.classList.toggle('active', respostaEstaVisivel)  
}
```

Nessa linha, acessamos as listas de classes do nosso cartão a partir da função **classList**. Após isso, utilizamos a função **toggle**, que é capaz de ativar ou desativar uma outra função. A função que queremos ativar é a **active**, que será controlada pelo valor armazenado na variável **respostaEstaVisivel**. Se o valor for **true**, a função é ativada; se o valor for **false**, ela é desativada.

Ao salvar e testar o projeto, veremos que o giro ainda não funciona.



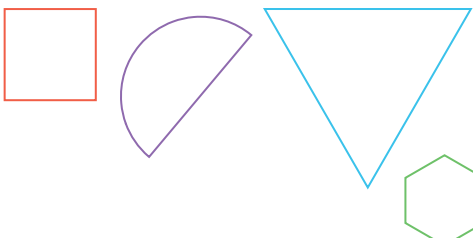
Esse erro ainda persiste porque precisamos avisar ao cartão que ele deverá estar atento à ação de clique. Para isso, adicionaremos um evento chamado **addEventListener**, que, em português, podemos traduzir como “adicionar um evento ouvinte”, que seria um evento capaz de “ouvir” ou perceber uma ação. O código ficará assim:

```
let respostaEstaVisivel = false

function viraCartao () {
  respostaEstaVisivel = !respostaEstaVisivel
  cartao.classList.toggle('active', respostaEstaVisivel)
}

cartao.addEventListener('click', viraCartao)
```

Perceba que a linha destacada foi adicionada fora da função **viraCartao**. Além disso, passamos dois parâmetros para esse evento: **click**, que informa o tipo de evento que ele deverá perceber, e **viraCartao**, que indica a ação a ser realizada quando ele perceber o evento de clique do mouse.



Agora, ao salvar e executar o projeto, veremos que os cartões viram de acordo com o clique do mouse:



Assim, finalizamos mais uma importante etapa do projeto. Na próxima aula, faremos as modificações finais!

► Desafio

Nesta aula, modificamos a forma como os nossos cartões giram, incluindo um evento de clique do mouse. Seu desafio é pesquisar as outras formas de utilizar a função **addEventListener**. Que outros tipos de eventos podem ser incluídos nessa função? Pesquise e registre suas descobertas no seu caderno, compartilhando-as posteriormente com seus colegas.



CLIQUE **AQUI** PARA AVALIAR ESTE MATERIAL